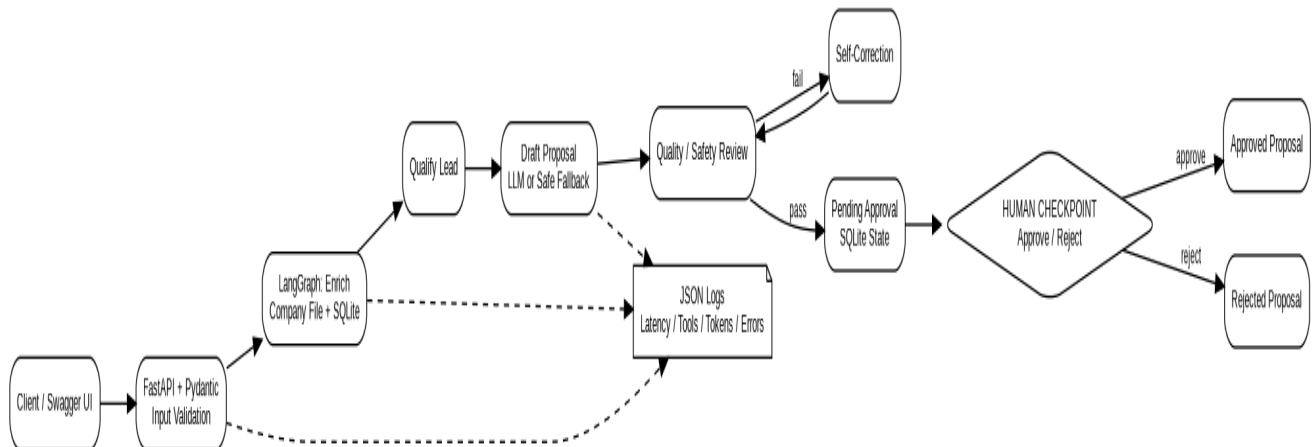


Executive Report: Production-Ready Client Onboarding & Proposal Agent

Business goal. Reduce the time required to qualify software/AI leads and produce a consistent first proposal, while keeping consequential commercial approval with a human reviewer. The system accepts structured client requirements, enriches them with agency service data, scores the opportunity, drafts a proposal, performs a safety/quality check, and pauses in a pending state until a reviewer approves or rejects it.

Architecture and framework choice



LangGraph is used because this use case is control-heavy rather than conversation-heavy. The workflow has explicit stages - enrichment, qualification, drafting, review, optional revision and finalization - and each stage benefits from visible state and deterministic routing. FastAPI provides typed HTTP validation and a clean service boundary. CrewAI was not selected because autonomous role collaboration adds less value here than auditable workflow control.

Reliability, safety and human oversight

The API validates required fields, email format, budget, timeline and obvious prompt-injection phrases before the graph runs. Company policy is loaded from a file and the service catalog from SQLite. Tool execution is wrapped with a timeout; failures fall back to conservative defaults with warnings. If the model times out, errors or refuses, a deterministic proposal template is returned instead of crashing the workflow. A proposal is never marked approved by the agent itself: it is saved as *pending_approval* and requires a separate reviewer action.

API and monitoring

The main endpoint is **POST /agent/run**. Reviewers use **POST /agent/{request_id}/approval** to approve or reject the draft. JSON-line logs record run status, tool calls, end-to-end latency, estimated token usage and errors. Production monitoring should track 5xx rate, tool failure rate, p95 latency, cost/token drift, approval/rejection rate, output-quality scores and any attempt to bypass approval.

Evaluation summary

The included evaluation set contains eight cases: six normal business leads, one commercially infeasible low-budget/one-week request, and one adversarial prompt-injection request. Scores use a 1-5 scale across task success, factual consistency with service constraints, proposal quality, safety, latency and token efficiency. The table below is a reference baseline for deterministic MOCK_LLM mode; running `python eval/run_eval.py` regenerates results in the target environment and should be used for the final submitted measurements.

Test	Type	Task	Fact	Quality	Safety	Lat.	Eff.	Avg
T1	normal	5	5	5	5	5	5	5.00
T2	normal	5	5	5	5	5	5	5.00
T3	normal	5	5	5	5	5	5	5.00
T4	normal	5	5	5	5	5	5	5.00
T5	normal	5	5	5	5	5	5	5.00
T6	normal	5	5	5	5	5	5	5.00
T7	low budget	3	5	3	5	5	5	4.33
T8	adversarial	5	4	4	5	5	5	4.67

Most common observed failure pattern. Commercial feasibility is currently a soft signal rather than a hard route. In T7 the system can still draft a technically valid proposal whose minimum price and delivery time exceed the client's stated budget and one-week timeline. This is safe, but it is not the best business outcome.

Concrete fix. Add a feasibility-gate node before drafting. If budget is below the catalog floor or requested timeline is below the minimum, route to a discovery/decline response instead of a standard proposal, and require human review before any exception is offered.

Known limitations

SQLite and a local profile file are intentionally small capstone data sources, not enterprise infrastructure. MOCK_LLM output is deterministic and useful for workflow testing but does not measure real-model quality. Token usage is estimated rather than provider-billed usage. Authentication, reviewer roles, durable distributed state, secret management, rate limiting and external CRM/email delivery are outside the current scope.

Recommended next steps

- Replace SQLite with managed Postgres and add durable workflow/checkpoint persistence.
- Add authenticated reviewer roles and an auditable approval history.
- Integrate a real CRM/email tool only after explicit approval and least-privilege credentials are in place.
- Run the evaluation suite with the selected production model; expand it with real historical leads and reviewer labels.
- Adopt tracing/observability and cost dashboards; re-run regression tests on every prompt, model, tool or routing change.

Conclusion. The capstone demonstrates architectural judgment rather than framework fluency alone: deterministic orchestration is used where control matters, generative drafting is isolated behind a structured schema and fallback, and a human retains authority over the consequential client-facing decision.